

Implicit Neural Representation

Foundations

Coordinate networks, continuous fields and implicit geometry

Matheus Levy de Lima Bessa

SIBGRAPI 2026 – Tutorial on Neural Unsigned Distance Fields

Meet the lecturers



Matheus Levy

Tecgraf Institute / PUC-Rio



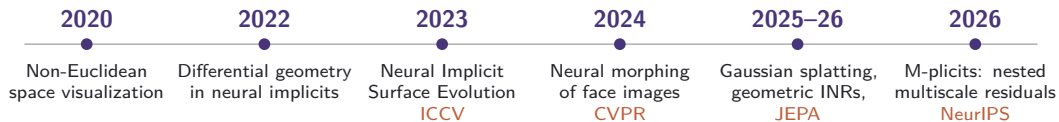
Vinícius da Silva

Tecgraf Institute

This tutorial grows from joint work with **Isabelle Rodrigues** and **Alberto Raposo**.
Thank you to the full survey team.

Some of our research

Our group works on **neural implicit representations** of geometry and images and, more recently, on latent world models. A short timeline:



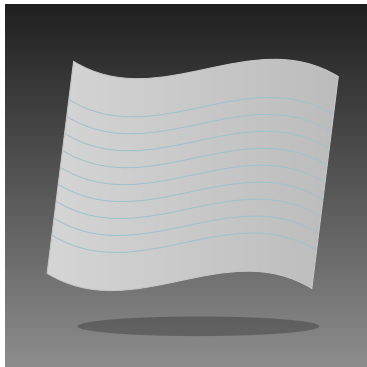
Takeaway. Implicit neural fields run through all of this work; M-plicits, our neural implicit surfaces accepted at NeurIPS 2026, is what brings us to today's topic.

Motivation: complex is easy, open is hard



✓ **Lucy statue: an SDF represents it**

Thin wings, a torch, fine drapery — but **closed** (watertight): every point is either inside or outside.



✗ **A sheet of paper: an SDF cannot**

Geometrically trivial — but **open**, with zero thickness: there is no inside, so the sign is undefined.

It is not about geometric complexity, it is about topology.

Slides ↔ notebook: how this tutorial runs

We alternate between these slides and a single notebook, `sibgrapi_udf_cpu_tutorial.ipynb`. Each notebook block is a **LAB**. Every LAB is announced by a ► **Now to the notebook** slide that names the block and the cells, and each LAB ends with a pointer back to the slide we resume from.

LAB	Notebook block	Slides
LAB 0	Setup — run it now (cells 2–3)	–
LAB 1	Analytic SDF and UDF	13–24
LAB 2	Gradients of the exact UDF	26–28
LAB 3	Your first neural field	33–38
LAB 4	Sampling from point clouds	40–46
LAB 5	Spatial gradients of a trained net	48–57

LAB	Notebook block	Slides
LAB 6	Gradient-based projection	59–64
LAB 7	ReLU vs. SIREN	66–72
LAB 8	Marching Cubes: SDF vs. UDF	74–78
LAB 9	Where the CPU time goes	80–85
LAB 10	End-to-end Neural UDF in 3D	87–94

Open the notebook and run **LAB 0** before we start: it only imports and seeds.

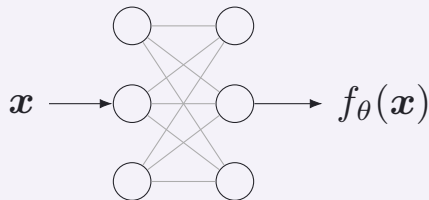
A shift in perspective

Discrete representation

1.7	1.1	0.7	0.5	0.7	1.1	1.7
1.1	0.3	-0.3	-0.5	-0.3	0.3	1.1
0.7	-0.3	-1.1	-1.5	-1.1	-0.3	0.7
0.5	-0.5	-1.5	-2.5	-1.5	-0.5	0.5
0.7	-0.3	-1.1	-1.5	-1.1	-0.3	0.7
1.1	0.3	-0.3	-0.5	-0.3	0.3	1.1
1.7	1.1	0.7	0.5	0.7	1.1	1.7

Each grid point stores a **sampled value** of the function.

Continuous implicit representation



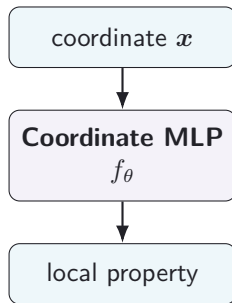
The network stores the **parameters of a function**, which can be queried at arbitrary coordinates.

Coordinate networks

A coordinate network takes the spatial coordinate itself as input:

$$\boldsymbol{x} \longrightarrow f_{\theta}(\boldsymbol{x}).$$

In the next slides, we will see four concrete fields: **image**, **occupancy**, **distance field** and **radiance field**.

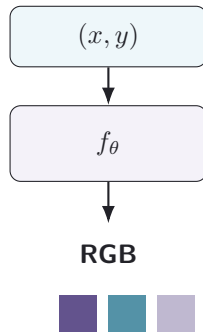


The same basic architecture can represent different fields. What changes is the **semantics of the output**.

Field 1: image

$$f_{\theta} : (x, y) \mapsto RGB \in [0, 1]^3$$

- ▶ Input: normalized **pixel** coordinate;
- ▶ Output: color (or intensity), not geometry;
- ▶ The image becomes a **continuous function**: it can be queried at any resolution, including in between the original pixels;
- ▶ A classic benchmark for studying the **spectral bias** of MLPs (SIREN, Fourier features).

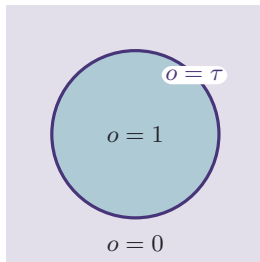


Each query (x, y) returns the local color – with no fixed pixel grid.

Field 2: occupancy

$$f_{\theta} : \mathbf{x} \mapsto o \in [0, 1]$$

- ▶ Output: **occupancy probability** (inside/outside);
- ▶ The surface is the **decision boundary** $o(\mathbf{x}) = \tau$; τ is an extraction hyperparameter (for example, 0.5 for a calibrated probabilistic classifier);
- ▶ A **classification** problem, not a regression one;
- ▶ It does not tell *how far* a point is from the surface, only which side it is on.

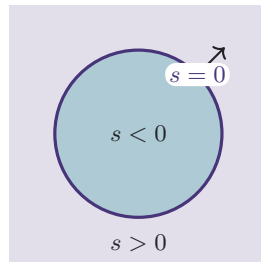


The network learns a continuous and smooth **spatial classification**.

Field 3: distance field

$$f_\theta : \mathbf{x} \mapsto s \in \mathbb{R} \quad (\text{or } u \geq 0)$$

- ▶ **SDF**: signed distance – $s < 0$ inside, $s > 0$ outside;
- ▶ **UDF**: unsigned distance – $u \geq 0$ on either side;
- ▶ The surface is the exact **zero level set**, not just a decision boundary;
- ▶ Away from singularities, $\|\nabla s\| = 1$ (the **Eikonal** property).

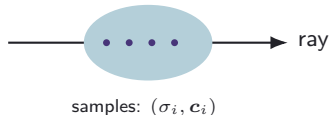


Distance is a **metric** measure, not just an inside/outside label.

Field 4: radiance field

$$f_{\theta} : (\mathbf{x}, \mathbf{d}) \mapsto (\sigma, \mathbf{c})$$

- ▶ Extra input: viewing direction $\mathbf{d}$;
- ▶ Output: density $\sigma \geq 0$ (local opacity) and color $\mathbf{c}$ that may vary with $\mathbf{d}$ (reflections, specularity);
- ▶ It does not define an explicit surface: it describes a **continuous volume**;
- ▶ Rendering by **integration along the ray** (volume rendering) – the basis of NeRF.



The final pixel color is a **weighted sum** of the samples along the ray.

Mildenhall et al., NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, ECCV 2020.

Why can a network represent a spatial function?

Approximation capacity

An MLP composes affine transformations and nonlinearities:

$$\mathbf{h}_{\ell+1} = \phi(W_{\ell}\mathbf{h}_{\ell} + \mathbf{b}_{\ell}), \quad \mathbf{h}_0 = \mathbf{x}.$$

Under the usual assumptions on the activation and on compact domains, MLPs enjoy **universal approximation** properties: with enough capacity, they can approximate broad classes of continuous functions.

But the bias matters

In practice, **representability** does not imply **ease of learning**.

- ▶ Standard MLPs favor smoother components.
- ▶ Fine details require high frequencies.
- ▶ Fourier features and SIREN change this bias.

The architecture determines which functions are easy to learn, not just which functions exist.

Cybenko, 1989; Hornik et al., 1989; Rahaman et al., ICML 2019; Tancik et al., NeurIPS 2020; Sitzmann et al., NeurIPS 2020.

Implicit geometry = level set of a field

A scalar field assigns a value to every point:

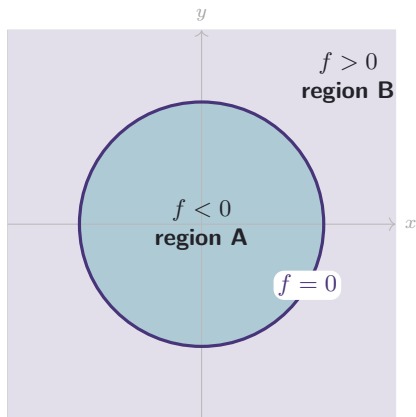
$$f : \mathbb{R}^3 \rightarrow \mathbb{R}.$$

A surface can be defined implicitly by

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^3 \mid f(\mathbf{x}) = c\}.$$

In neural implicit geometry, most often:

$$\boxed{\mathcal{S} = \{\mathbf{x} \mid f_{\theta}(\mathbf{x}) = 0\}}.$$



For a **signed field**, $f < 0$ and $f > 0$ can represent two sides (by convention, inside/outside). For a generic scalar field, these regions do not necessarily carry that semantics.

Three fundamental geometric fields

Field	Output	Surface	Information
Occupancy	$o(\mathbf{x}) \in [0, 1]$	decision boundary	inside / outside
SDF	$s(\mathbf{x}) \in \mathbb{R}$	$s(\mathbf{x}) = 0$	distance + sign
UDF	$u(\mathbf{x}) \geq 0$	$u(\mathbf{x}) = 0$	unsigned distance

Occupancy

$$o(\mathbf{x}) \approx P(\text{inside} \mid \mathbf{x})$$

Spatial classification.

SDF

$$s(\mathbf{x}) = \pm \min_{\mathbf{p} \in S} \|\mathbf{x} - \mathbf{p}\|_2$$

Signed metric regression.

UDF

$$u(\mathbf{x}) = \min_{\mathbf{p} \in S} \|\mathbf{x} - \mathbf{p}\|_2$$

Distance without inside/outside.

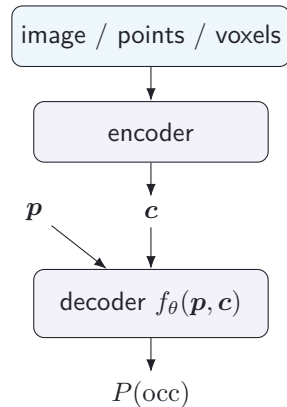
Occupancy Networks: geometry as continuous classification

$$f_{\theta}(\mathbf{p}, \mathbf{c}) \rightarrow [0, 1]$$

- ▶ $\mathbf{p}$: query point in $\mathbb{R}^3$;
- ▶ $\mathbf{c}$: code conditioned on the observation;
- ▶ output: occupancy probability.

The surface is a decision boundary:

$$\mathcal{S} = \{\mathbf{p} \mid f_{\theta}(\mathbf{p}, \mathbf{c}) = \tau\}.$$



Mescheder et al., CVPR 2019.

DeepSDF: geometry as signed distance

$$f_{\theta}(z_i, \mathbf{x}) \rightarrow s$$

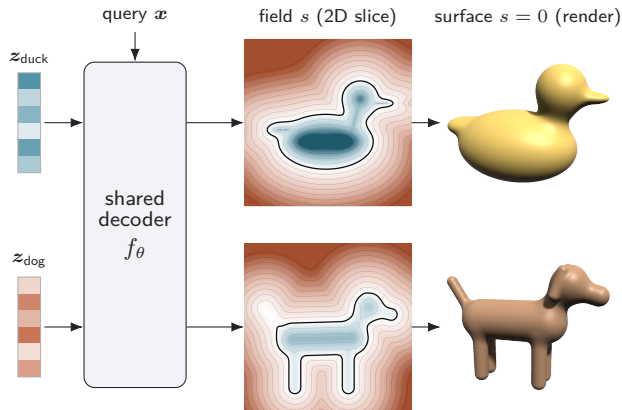
- $\mathbf{x}$: query coordinate;
- z_i : latent code of shape i ;
- s : signed distance.

$$\mathcal{S}_i = \{\mathbf{x} \mid f_{\theta}(z_i, \mathbf{x}) = 0\}.$$

Auto-decoder: the latent codes are variables optimized jointly with the network.

Same weights θ , different $z_i \Rightarrow$
different field $\Rightarrow$ different surface.

Park et al., CVPR 2019.



Occupancy Networks × DeepSDF

Aspect	Occupancy Networks	DeepSDF
Output	occupancy probability	signed distance
Local problem	classification	regression
Surface	decision boundary	zero level set
Metric information	indirect	explicit
Inside/outside	yes	yes, through the sign
Conditioning	encoder → code	latent code per shape

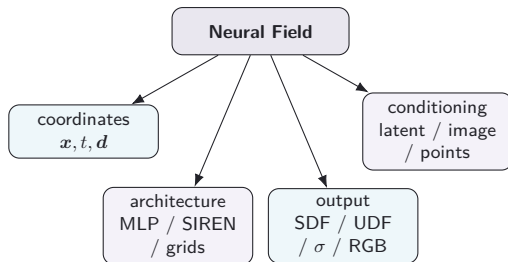
Occupancy asks

“Is this point inside?”

SDF asks

“What is the distance, and which side am I on?”

Neural fields: a broader view



- **SIREN**: periodic activations and well-behaved spatial derivatives;
- **Fourier features**: make high frequencies easier to fit;
- **NeRF**: density and radiance, not only surfaces;
- fields can be spatial, temporal or spatio-temporal.

“Neural field” is a representation paradigm, not a single architecture.

SIREN: Sitzmann et al., NeurIPS 2020. Fourier features: Tancik et al., NeurIPS 2020. NeRF: Mildenhall et al., ECCV 2020.

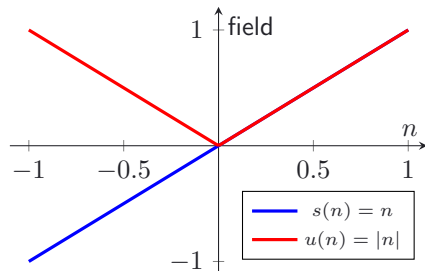
Bridge to UDFs: removing the sign changes the problem

Near a smooth surface, along the normal coordinate n :

$$s(n) \approx n, \quad u(n) = |s(n)| \approx |n|.$$

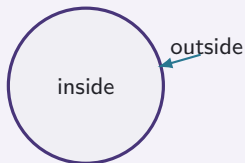
- **Gain:** it does not require a globally consistent inside/outside.
- **Cost:** a cusp appears at the zero level set.

$$\frac{d}{dn}|n| = \begin{cases} -1, & n < 0, \\ +1, & n > 0. \end{cases}$$



Why unsigned distance?

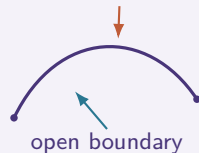
Closed, watertight surface



A signed field can rely on a global partition:

inside | outside.

Open / non-watertight surface



A global binary sign with geometric meaning may not exist.

Keep the distance; drop the sign.

UDFs naturally represent **open, non-watertight** surfaces with arbitrary topology, without requiring a global inside/outside convention.

Chibane, Mir, and Pons-Moll, "Neural Unsigned Distance Fields for Implicit Function Learning," NeurIPS 2020.

Unsigned Distance Field: definition

For a target surface $\mathcal{S} \subset \mathbb{R}^3$, the UDF is defined as

$$u_{\mathcal{S}}(\mathbf{x}) = \inf_{\mathbf{p} \in \mathcal{S}} \|\mathbf{x} - \mathbf{p}\|_2$$

Therefore,

$$u_{\mathcal{S}}(\mathbf{x}) \geq 0 \quad \forall \mathbf{x} \in \mathbb{R}^3.$$

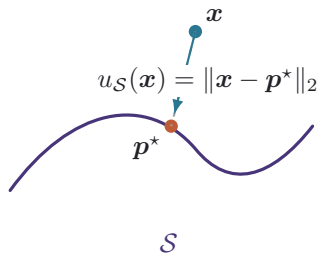
The surface is the **zero level set**:

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^3 \mid u_{\mathcal{S}}(\mathbf{x}) = 0\}.$$

What changed with respect to the SDF?

The metric information remains, but the global orientation encoded by the sign is removed.

Chibane et al., NeurIPS 2020.

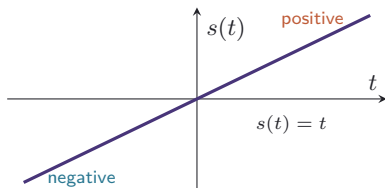


Each query answers only:

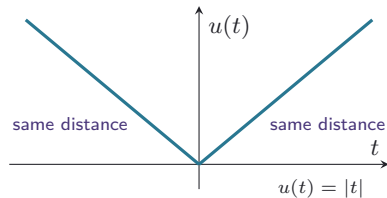
“What is the distance to the closest point on the surface?”

What is lost when the sign disappears?

Signed Distance Field



Unsigned Distance Field



$t < 0 \Rightarrow$ one side, $t > 0 \Rightarrow$ the other side.

$$u(-t) = u(t).$$

The UDF preserves proximity, but removes the information about which side of the surface.

Near the surface: the UDF behaves like an absolute value

Let $\mathbf{p} \in \mathcal{S}$ be a smooth surface point with unit normal $\mathbf{n}$, and consider a displacement along the normal coordinate:

$$\mathbf{x}(t) = \mathbf{p} + t\mathbf{n}.$$

Locally,

$$s_{\mathcal{S}}(\mathbf{x}(t)) \approx t,$$

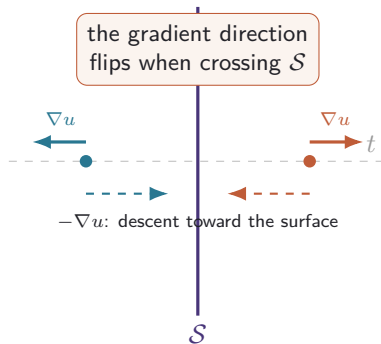
$$u_{\mathcal{S}}(\mathbf{x}(t)) \approx |t|.$$

Therefore,

$$\frac{d}{dt}|t| = \begin{cases} -1, & t < 0, \\ +1, & t > 0. \end{cases}$$

At $t = 0$, the classical derivative is **undefined**.

Local model: $u(\mathbf{p} + t\mathbf{n}) = |t|$. Chibane et al., NeurIPS 2020; see also the companion survey of this tutorial (to appear).



UDFs: what we gain and what we lose

What we gain

- ▶ No global inside/outside partition is required.
- ▶ Natural support for open boundaries.
- ▶ Compatible with non-watertight geometry.
- ▶ Keeps the zero level set interpretation:

$$\mathcal{S} = \{x \mid u(x) = 0\}.$$

What we lose

- ▶ There is no sign transition across the surface.
- ▶ Points on opposite sides can have the same value.
- ▶ The ideal field has a cusp at the surface.
- ▶ Methods based on sign changes are no longer directly applicable.

$$\text{SDF } s(t) = t \implies \text{UDF } u(t) = |t|$$

Next topic: if the ideal UDF is not differentiable at zero, what does the neural network learn in that region?

Chibane, Mir, and Pons-Moll, NeurIPS 2020.

Now to the notebook

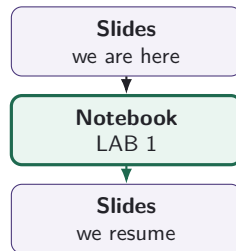
▶ LAB 1 — Analytic SDF and UDF

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 1 cells 4–11

What we run there

- ▶ the circle SDF and its zero level set;
- ▶ the same field with the sign removed, $u = |s|$;
- ▶ the local profile $u(t) = |t|$ and its cusp.



↩ Back to slide 26

The problem originates in the UDF itself

The problem originates in the UDF itself

Near a smooth point $p \in \mathcal{S}$, consider

$$x(t) = p + tn.$$

The ideal UDF behaves locally as

$$u_{\mathcal{S}}(x(t)) = |t|.$$

Hence, the one-sided derivatives are

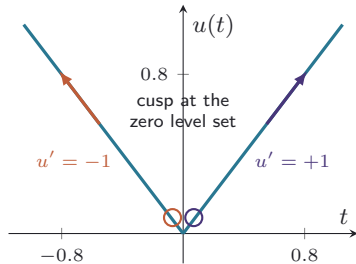
$$\lim_{t \rightarrow 0^-} \frac{d}{dt}|t| = -1, \quad \lim_{t \rightarrow 0^+} \frac{d}{dt}|t| = +1.$$

Consequence

There is no unique classical derivative at $t = 0$.

The gradient direction **flips** when crossing the surface.

For an exact UDF, the local model $u(p + tn) = |t|$ yields opposite one-sided derivatives at the zero level set.



But the neural network has to learn an approximation

A neural network learns

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0}, \quad f_{\theta}(x) \approx u_{\mathcal{S}}(x).$$

The key point is that

low error in $f_{\theta}(x) \not\Rightarrow$ correct gradient $\nabla_x f_{\theta}(x)$.

If the approximation is smooth

If the surrogate is differentiable, non-negative and attains $f_{\theta} = 0$ at an interior point, that point is a local minimum; therefore

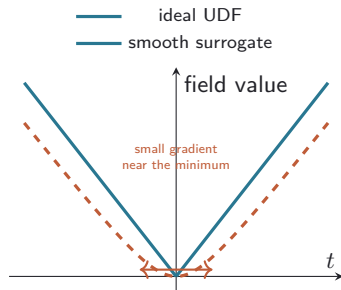
$$\nabla_x f_{\theta}(x) = 0.$$

So smoothing the kink does not preserve $\|\nabla f\| = 1$ exactly at the zero level set.

If the approximation changes rapidly

The gradient direction may become unstable or inconsistent between neighboring points.

A neural UDF approximates the scalar field; a low distance error does not guarantee reliable gradients.



The network can get the *distances* quite right and still provide poor derivatives.

Why does the gradient near the surface matter?

In NDF, the learned gradient takes part in projecting points toward the zero level set. A conceptual form of the step is

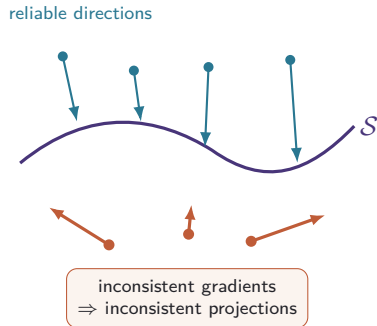
$$\mathbf{x}_{k+1} = \mathbf{x}_k - f_{\theta}(\mathbf{x}_k) \frac{\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}_k)}{\|\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}_k)\| + \varepsilon}$$

- ▶ $f_{\theta}(\mathbf{x})$ estimates **how far** to move;
- ▶ $\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$ estimates **in which direction** to move;
- ▶ the descent direction is $-\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$.

If the gradient fails

Nearby points may be projected to inconsistent positions, producing holes, fragments or a noisy surface.

NDF uses the distance and the gradient of the learned field to project samples toward the surface.



Now to the notebook

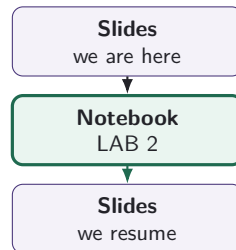
▶ LAB 2 — Gradients of the exact UDF

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 2 cells 12–16

What we run there

- ▶ `torch.autograd.grad` on the analytic UDF;
- ▶ gradient norms and the sign flip across the surface;
- ▶ why $\|\nabla u\| = 1$ holds away from the cusp.



↩ Back to slide 30

Two later answers to the problem

Two later answers to the problem

Level-Set UDF ICCV 2023

Idea: do not rely directly on the hardest region alone, the zero level set.

- ▶ uses a **positive level set** near the surface;
- ▶ this level set is geometrically more regular;
- ▶ projects it onto the zero level set;
- ▶ aligns gradients across the level sets.



Transfer information from a more stable region to the surface.

Zhou et al., Level-Set UDF, ICCV 2023.

Fainstein et al., DUDF, CVPR 2024.

DUDF CVPR 2024

Idea: modify the representation itself to avoid the non-differentiable kink of the canonical UDF.

- ▶ introduces a **differentiable reparametrization**;
- ▶ uses **hyperbolic scaling** near the zero level set;
- ▶ enables optimization with more stable gradients;
- ▶ combines the representation with tailored field constraints.



Change the field in order to change its differential behavior.

The main message

Correct distance is not enough

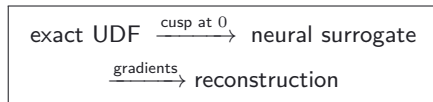
For projection-based reconstruction, we want simultaneously

$$f_{\theta}(\mathbf{x}) \approx u(\mathbf{x})$$

and a geometrically useful derivative field:

$$\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}) \text{ stable and coherent near } \mathcal{S}.$$

The problem can be summarized as



Next topic: how do we turn this into a trainable neural UDF, $f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0}$, with distance regression?

NDF: Chibane et al., NeurIPS 2020. Level-Set UDF: Zhou et al., ICCV 2023.
DUDF: Fainstein et al., CVPR 2024.

Mathematical problem

$u(t) = |t|$ is not differentiable at 0



Learning problem

the surrogate may flatten or distort the gradient



Geometric problem

projection and extraction become unstable

Takeaways

1. INRs represent signals as **neural functions of coordinates**.
2. Surfaces can emerge as **level sets** of scalar fields.
3. Occupancy Networks learns a **continuous decision boundary**.
4. DeepSDF learns a shape-conditioned **signed distance field**.
5. Neural fields generalize the idea to density, radiance, dynamics and other signals.
6. UDFs remove the sign and widen the class of surfaces, but introduce a **singularity at the zero level set**.

Next step

Neural UDFs in practice: distance regression, sampling, autograd, projection and reconstruction.

From the geometric UDF to a neural UDF

The exact UDF is

$$u_{\mathcal{S}}(\mathbf{x}) = \inf_{\mathbf{p} \in \mathcal{S}} \|\mathbf{x} - \mathbf{p}\|_2.$$

Conceptually, a neural UDF approximates

$$u_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0}, \quad u_{\theta}(\mathbf{x}) \approx u_{\mathcal{S}}(\mathbf{x}).$$

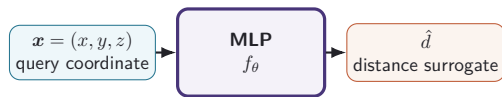
Its surface is

$$\mathcal{S}_{\theta} = \{\mathbf{x} \in \mathbb{R}^3 \mid u_{\theta}(\mathbf{x}) = 0\}.$$

Didactic baseline for the next slides

We will use a scalar surrogate $f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}$ trained with UDF targets. A linear output **does not guarantee** $f_{\theta} \geq 0$; for a strict UDF, non-negativity must be enforced by the parametrization.

Chibane, Mir, and Pons-Moll, NeurIPS 2020.



The distance semantics comes from the **supervision**; non-negativity is an additional decision of the representation.

Supervision: distance regression

The training set contains pairs

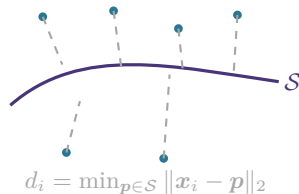
$$(\mathbf{x}_i, d_i), \quad d_i = u_S(\mathbf{x}_i).$$

For each query point:

$$\mathbf{x}_i \longrightarrow f_\theta(\mathbf{x}_i) = \hat{d}_i.$$

The task is a **supervised scalar regression**:

$$\hat{d}_i \approx d_i.$$



What is not supervised directly

- ▶ surface connectivity;
- ▶ mesh;
- ▶ normals;
- ▶ spatial gradients.

Original NDF: the field is conditioned on the sparse observation X ; the distance supervision is the UDF of the target surface S .

The L1 objective and the truncated NDF loss

Didactic baseline

For direct regression of a single shape:

$$\mathcal{L}_{L1} = \frac{1}{N} \sum_{i=1}^N |f_{\theta}(\mathbf{x}_i) - d_i|.$$

This turns the problem into pairs

coordinate $\rightarrow$ distance.

Chibane, Mir & Pons-Moll, NeurIPS 2020, Sec. 3.2.

Original NDF

NDF is conditioned on the observation X and uses truncated regression:

$$\mathcal{L} = \sum_{X \in \mathcal{B}} \sum_{p \in \mathcal{P}} |\min(f_X(p), \delta) - \min(\text{UDF}(p, S_X), \delta)|.$$

The clamp concentrates capacity in the neighborhood of the surface; the paper uses $\delta = 10$ cm in its experiments.

Batching coordinates

An MLP receives

$$\mathbf{X} \in \mathbb{R}^{B \times 3},$$

with

$$\mathbf{X} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_B & y_B & z_B \end{bmatrix}.$$

The targets are

$$\mathbf{d} \in \mathbb{R}^{B \times 1},$$

and the network produces

$$f_{\theta}(\mathbf{X}) \in \mathbb{R}^{B \times 1}.$$



We do **not** evaluate one coordinate at a time.
We process thousands of queries in parallel.

batching $\Rightarrow$ vectorized training on the GPU

Minimal PyTorch model: distance surrogate

```
class NeuralUDF(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(3, 128), nn.ReLU(),  
            nn.Linear(128, 128), nn.ReLU(),  
            nn.Linear(128, 1)  
        )  
  
    def forward(self, x):  
        return self.net(x)
```

```
pred = model(xyz)                # [B, 1]  
loss = torch.abs(pred - udf).mean()  
  
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

Input

$xyz.shape = [B, 3]$

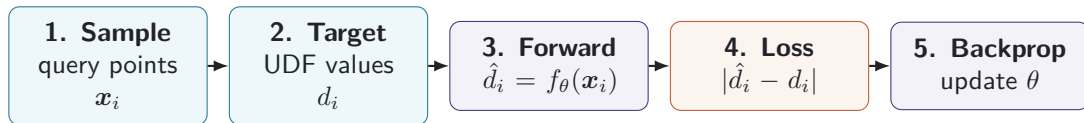
Output

$pred.shape = [B, 1]$

Minimal version

Linear output: it can be negative. A didactic surrogate; a strict UDF needs non-negative parametrization, and the original NDF uses a non-negative decoder conditioned on X .

One training iteration



The formulation in one line

$$\mathbf{X} \in \mathbb{R}^{B \times 3} \xrightarrow{f_\theta} \hat{\mathbf{d}} \in \mathbb{R}^{B \times 1} \xrightarrow{\mathcal{L}_1} \text{update } \theta$$

Next topic: where do the pairs (x_i, d_i) come from?
Surface samples, near-surface perturbations and spatial samples.

Chibane, Mir, and Pons-Moll, Neural Unsigned Distance Fields for Implicit Function Learning, NeurIPS 2020.

Now to the notebook

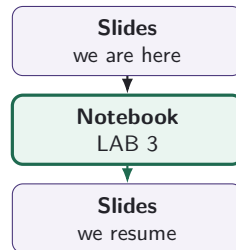
▶ LAB 3 — Your first neural field

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 3 cells 17–23

What we run there

- ▶ a TinyMLP trained on signed targets;
- ▶ the learned zero level set;
- ▶ the same network trained on unsigned targets.



↩ Back to slide 40

Where do the pairs (x_i, d_i) come from?

Where do the pairs $(\mathbf{x}_i, d_i)$ come from?

In the previous topic, we trained with

$$(\mathbf{x}_i, d_i), \quad d_i = u_S(\mathbf{x}_i).$$

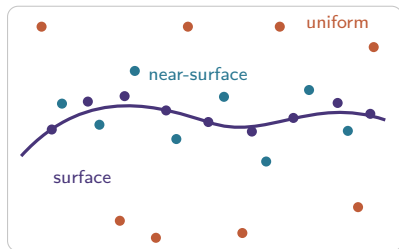
In the **tutorial baseline**, the pairs come from a point cloud. In the original NDF, the queries are sampled near the ground-truth geometry and the UDF is computed with respect to the target surface S .

A useful dataset mixes three regimes:

surface + near-surface + spatial

- ▶ points exactly on the surface;
- ▶ points perturbed near it;
- ▶ points sampled uniformly in the volume.

Baseline: point-set targets. Original NDF: surface-UDF ground truth.



The goal is to teach both the **local** behavior of the field and its **global** structure.

1. Surface samples

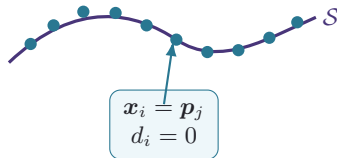
We start from points of the point cloud:

$$\mathbf{p}_j \in \mathcal{P}.$$

If $\mathbf{p}_j$ is a sample of the surface, we use

$$\mathbf{x}_i = \mathbf{p}_j \implies d_i \approx 0.$$

For a noisy point cloud, this only approximates the zero level set.



What do these samples teach?

They explicitly anchor the zero level set:

$$f_{\theta}(\mathbf{p}_j) \approx 0.$$

But they are not enough

Training only with $d = 0$ says nothing about how the field grows away from the surface.

2. Near-surface perturbations

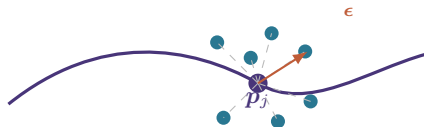
Starting from a surface point p_j , we create a perturbed query:

$$\mathbf{x}_i = \mathbf{p}_j + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$$

or another small-magnitude noise.

These queries concentrate supervision where the geometry matters most:

$$d_i = u_{\mathcal{S}}(\mathbf{x}_i).$$



higher query density
in the near-surface region

Why do they matter?

- ▶ they define the shape of the field near $u = 0$;
- ▶ they improve the resolution of geometric detail;
- ▶ they provide samples on both sides without needing a sign.

3. Uniform spatial samples

We also sample the spatial domain:

$$\mathbf{x}_i \sim \mathcal{U}(\Omega),$$

where Ω is, for example, a normalized bounding box.

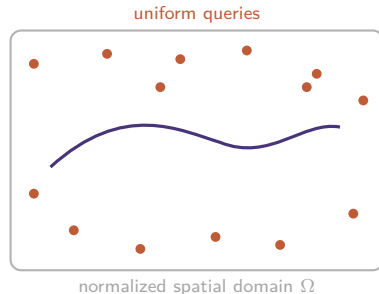
These points provide **global** context:

- ▶ they prevent the network from learning only a narrow band;
- ▶ they teach distances far from the surface;
- ▶ they help define the field over the whole volume.

A practical strategy is to mix:

$$B = B_{\text{surf}} + B_{\text{near}} + B_{\text{uniform}}.$$

near-surface accuracy + global field coverage



Nearest-neighbor distance targets

If the surface is observed only as a point cloud

$$\mathcal{P} = \{\mathbf{p}_j\}_{j=1}^M,$$

a practical approximation of the UDF is

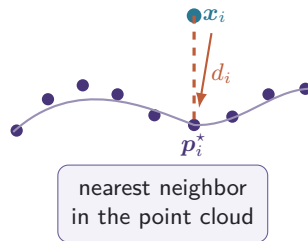
$$d_i = \min_{\mathbf{p}_j \in \mathcal{P}} \|\mathbf{x}_i - \mathbf{p}_j\|_2$$

For each query:

$$\mathbf{x}_i \longrightarrow \mathbf{p}_i^* \longrightarrow d_i.$$

Geometric approximation

This target measures the distance to the *point set*; its quality depends on density and noise. **It is not the target of the original NDF**, which uses the ground-truth UDF of the target surface.



Practical dataset generation in Python

```
def make_queries(points, n_near, n_uniform, sigma=0.01):  
    # near-surface samples  
    ids = torch.randint(len(points), (n_near,))  
    surface = points[ids]  
    near = surface + sigma * torch.randn_like(surface)  
  
    # uniform samples in normalized box  $[-1, 1]^3$   
    uniform = 2.0 * torch.rand(n_uniform, 3) - 1.0  
  
    xyz = torch.cat([surface, near, uniform], dim=0)  
  
    # tutorial approximation: distance to nearest input point  
    dist = torch.cdist(xyz, points)  
    udf = dist.min(dim=1).values[:, None]  
  
    return xyz, udf
```

Shapes

$\text{points} : [M, 3]$

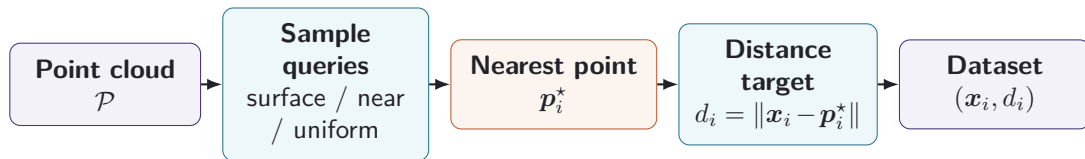
$\text{xyz} : [B, 3]$

$\text{udf} : [B, 1]$

Research scale

`torch.cdist` is great for a demonstration, but it can be expensive for large point clouds. KD-trees, chunking or specialized kernels are practical alternatives.

Didactic dataset generation pipeline



The goal of sampling

show the network where the surface is + how the distance grows around it

Next topic: now that we have x_i , let us enable `requires_grad` and compute $\nabla_x f_\theta(x)$ with PyTorch autograd.

Tutorial baseline: nearest-point targets are a pedagogical approximation; the original NDF uses ground-truth surface UDF supervision.

Now to the notebook

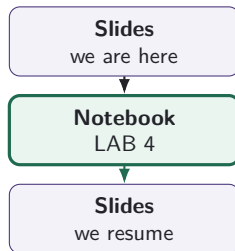
▶ LAB 4 — Sampling from point clouds

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 4 cells 24–30

What we run there

- ▶ surface / near-surface / uniform queries;
- ▶ nearest-point distance targets;
- ▶ a neural UDF of an *open* curve.



↩ Back to slide 48

What is a spatial gradient?

What is a spatial gradient?

For a neural scalar field

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R},$$

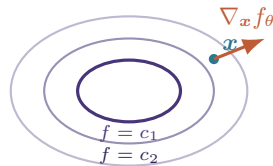
the gradient with respect to the **spatial input** is

$$\nabla_x f_{\theta}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_{\theta}}{\partial x} \\ \frac{\partial f_{\theta}}{\partial y} \\ \frac{\partial f_{\theta}}{\partial z} \end{bmatrix}$$

It points in the direction of steepest local increase of f_{θ} .

For distance fields, the gradient provides:

- ▶ the local normal direction to the level set;
- ▶ in a UDF, the orientation flips between sides;
- ▶ outside the cut locus, the direction for projecting to the closest point.



the gradient is perpendicular to the local level set

First step: enabling gradients on the input

By default, a coordinate tensor does not require derivatives.

```
xyz = xyz.to(device)
xyz.requires_grad_(True)
```

```
pred = model(xyz)
```

PyTorch now records the operations connecting

$$\text{xyz} \longrightarrow f_{\theta}(\text{xyz}).$$

A robust way to create a fresh differentiable input is

```
xyz = xyz.detach().clone()
xyz = xyz.requires_grad_(True)
```

The key point

We want to differentiate with respect to the **coordinates**, not only with respect to the weights:

$$\frac{\partial f_{\theta}}{\partial \mathbf{x}}.$$

Two kinds of gradient

$$\nabla_{\theta} f_{\theta} \neq \nabla_{\mathbf{x}} f_{\theta}$$

Training backpropagation usually updates θ .

Here we also want derivatives with respect to $\mathbf{x}$.

torch.autograd.grad: the minimal pattern

For a batch of queries:

```
xyz = xyz.detach().requires_grad_(True)
pred = model(xyz)                # [B, 1]
```

```
grad = torch.autograd.grad(
    outputs=pred,
    inputs=xyz,
    grad_outputs=torch.ones_like(pred),
    create_graph=True
)[0]                             # [B, 3]
```

Then

$$\text{grad}[i] = \nabla_{\mathbf{x}_i} f_{\theta}(\mathbf{x}_i).$$

outputs

The predicted scalar field:

$$[B, 1].$$

inputs

The coordinates:

$$[B, 3].$$

grad_outputs

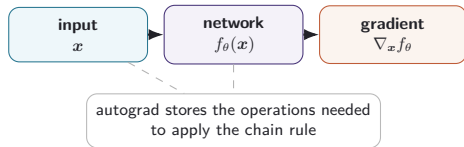
Since there are B outputs, we provide the weight vector

$$\mathbf{1}.$$

$$[B, 3] \xrightarrow{f_{\theta}} [B, 1] \xrightarrow{\text{autograd.grad}} [B, 3]$$

For a pointwise MLP, each $f_{\theta}(\mathbf{x}_i)$ depends on the query $\mathbf{x}_i$; each returned row corresponds to the spatial gradient of that sample.

The computational graph and create_graph



`create_graph=False`

The gradient is computed, but we do not need to differentiate it again. Useful for analysis or inference.

`create_graph=True`

The gradient itself stays in the graph. Required when a loss depends on

$$\nabla_x f_\theta(x),$$

for example:

$$(\|\nabla_x f_\theta\|_2 - 1)^2.$$

Cost

Higher-order derivatives increase memory usage.

Leaf tensors: a common source of confusion

A **leaf tensor** is a tensor that was not created as the result of an operation tracked by autograd.

A safe example:

```
xyz = batch_xyz.to(device)
xyz = xyz.detach().clone()
xyz.requires_grad_(True)

print(xyz.is_leaf)    # True
```

After `backward()`, the `.grad` field is populated by default for **leaf tensors** that require gradients.

autograd.grad

For spatial gradients, we usually prefer:

```
grad = torch.autograd.grad(
    pred, xyz, ...
)[0]
```

It **returns** the gradient directly.

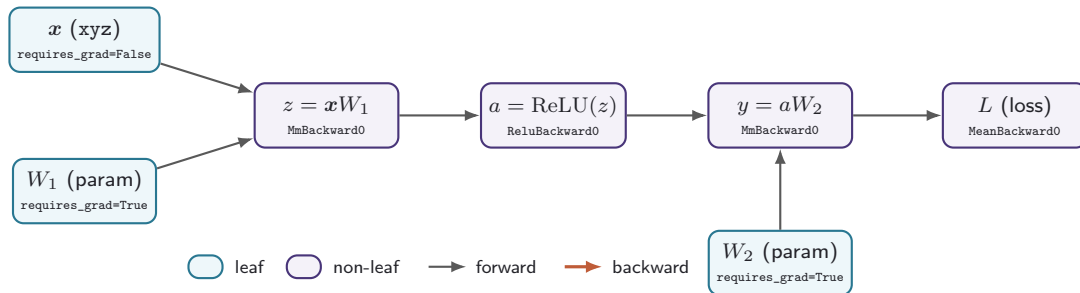
Intermediate tensor?

To inspect `.grad` of a non-leaf tensor after `backward()`, use:

```
tensor.retain_grad()
```

before the backward pass.

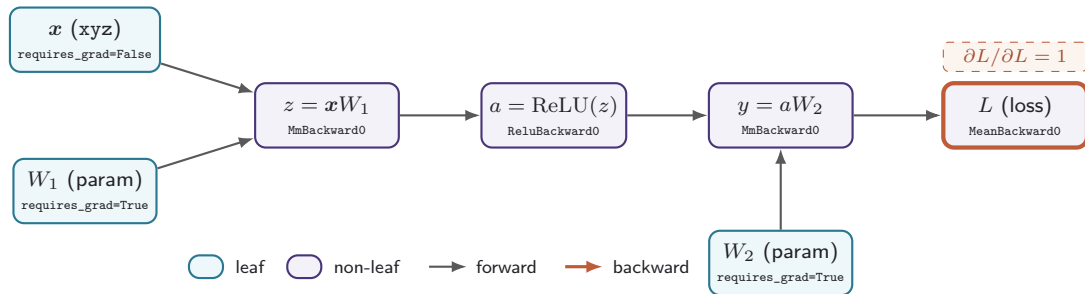
Training: which gradients are stored, which are discarded



1. Training forward pass: the graph is recorded

Each tracked operation creates a **non-leaf** tensor with a `grad_fn`. The **leaves** have no history: the parameters W_1 , W_2 (`requires_grad=True`) and the input batch xyz, which is plain data (`requires_grad=False`).

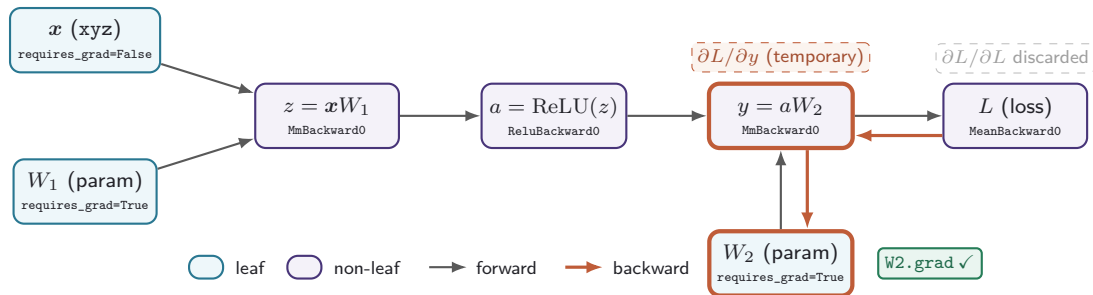
Training: which gradients are stored, which are discarded



2. `loss.backward()` starts at the root

Autograd seeds $\partial L / \partial L = 1$ and walks the graph in reverse, applying the local derivative of each `grad_fn` (chain rule).

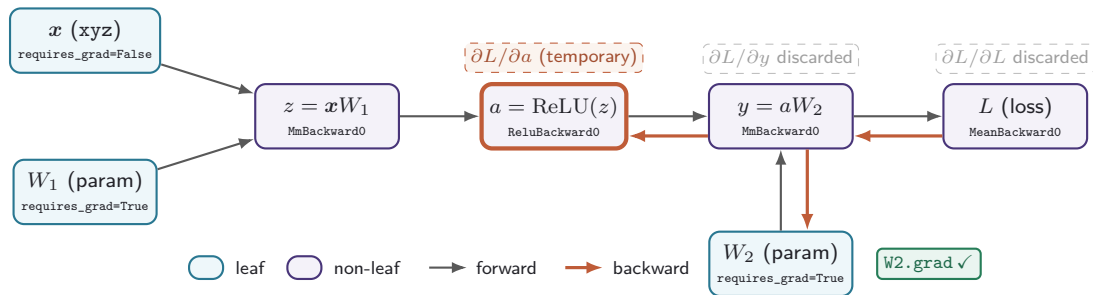
Training: which gradients are stored, which are discarded



3. Through $y = aW_2$: the first leaf is reached

MmBackward0 receives $\partial L / \partial y$ and computes $\partial L / \partial W_2 = a^\top \partial L / \partial y$. W_2 is a leaf, so the result is **accumulated** in $W_2.\text{grad}$.

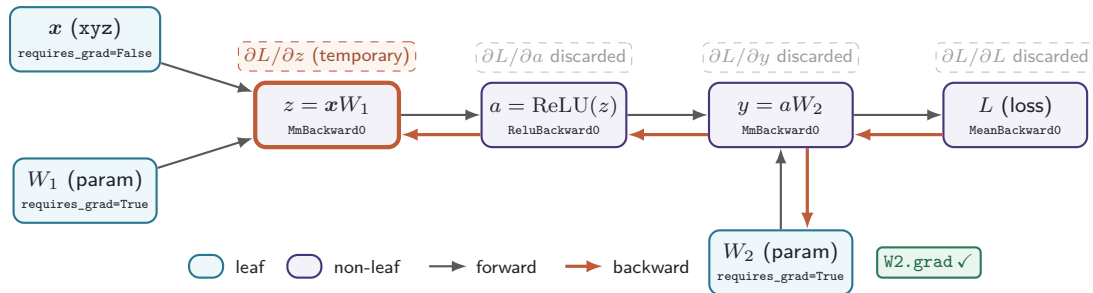
Training: which gradients are stored, which are discarded



4. Intermediate gradients are temporary

$\partial L / \partial a = \partial L / \partial y W_2^\top$ is passed on. Once consumed, $\partial L / \partial y$ is **discarded**: y is not a leaf, so nothing is written to $y.\text{grad}$.

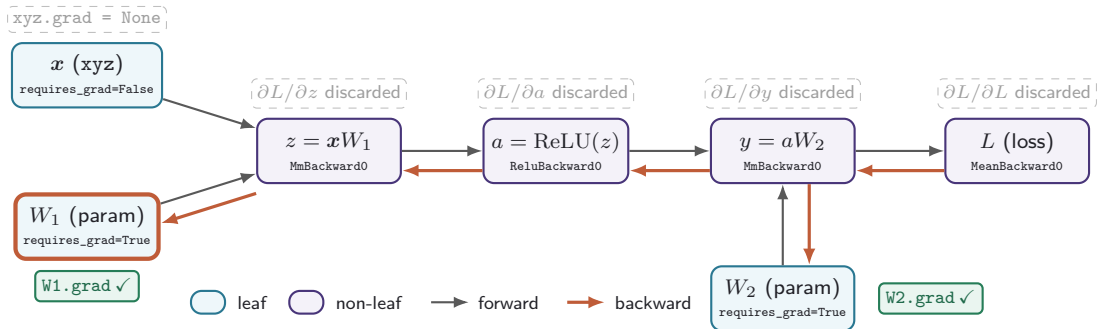
Training: which gradients are stored, which are discarded



5. Through the ReLU

$\partial L/\partial z = \partial L/\partial a \odot \mathbb{I}[z > 0]$. The gradient of the activation, $\partial L/\partial a$, is discarded as well.

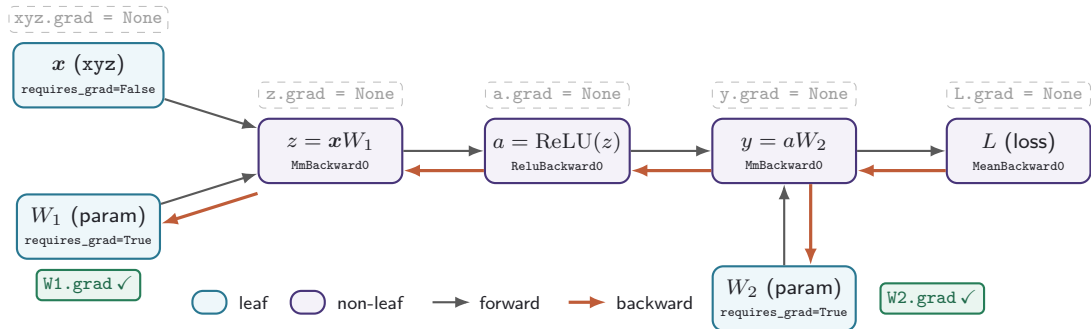
Training: which gradients are stored, which are discarded



6. Reaching the remaining leaves

$\partial L / \partial W_1 = x^\top \partial L / \partial z$ is accumulated in `W1.grad`. `xyz` is also a leaf, but it does not require gradients: autograd never follows that edge, so nothing is computed for it.

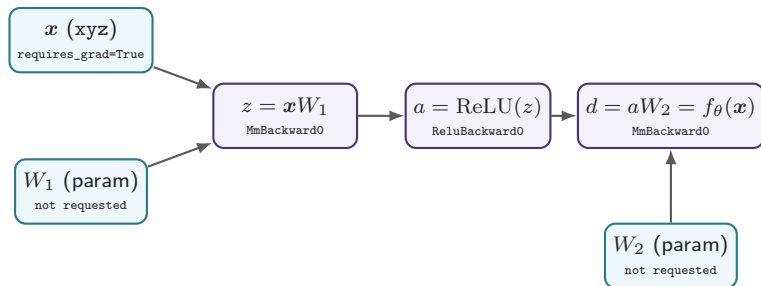
Training: which gradients are stored, which are discarded



Result after backward()

`.grad` is kept only on the **parameters**, where `optimizer.step()` reads it (then `zero_grad()`, since `.grad` accumulates). z , a , y , L keep nothing and the graph is freed. An Eikonal loss would also need `xyz.requires_grad_(True)`.

Inference: the gradient of the input leaf xyz



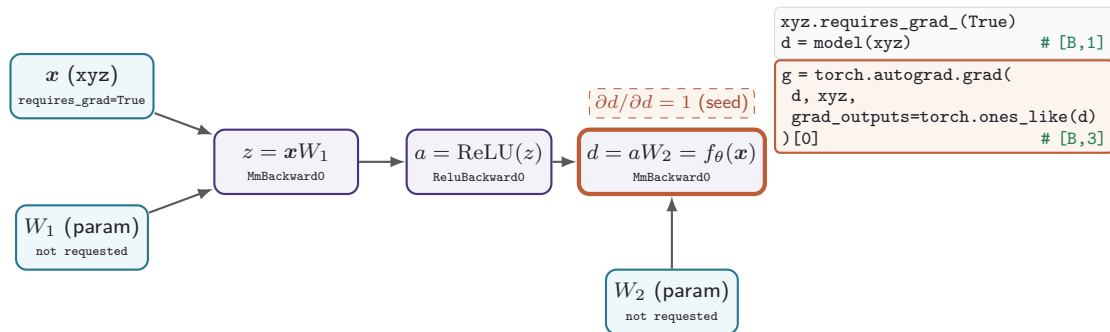
```
xyz.requires_grad_(True)
d = model(xyz) # [B,1]

g = torch.autograd.grad(
    d, xyz,
    grad_outputs=torch.ones_like(d)
)[0] # [B,3]
```

1. Inference forward pass: now xyz is tracked

`requires_grad_(True)` makes the input a leaf that autograd tracks. Autograd must stay **on**: under `torch.no_grad()` or `inference_mode()` no graph is recorded, and there is nothing to differentiate.

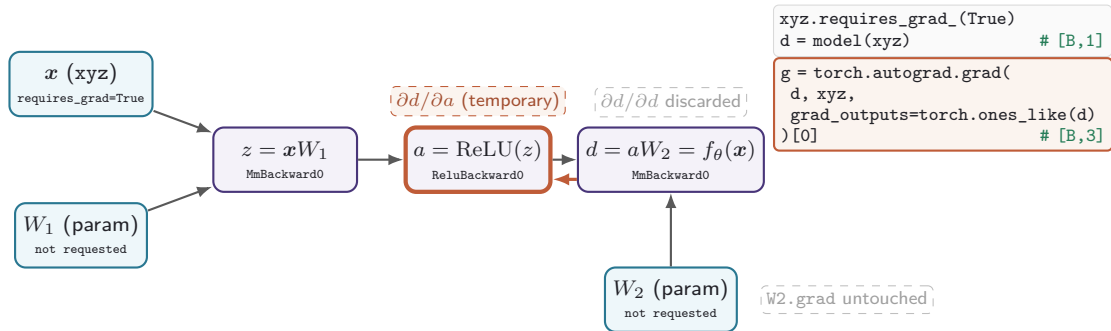
Inference: the gradient of the input leaf xyz



2. `autograd.grad(d, xyz)` starts at the distance

The root is the predicted distance d , not a loss. d has B values, so `grad_outputs=ones_like(d)` gives the seed $\partial d / \partial d = 1$ for each point.

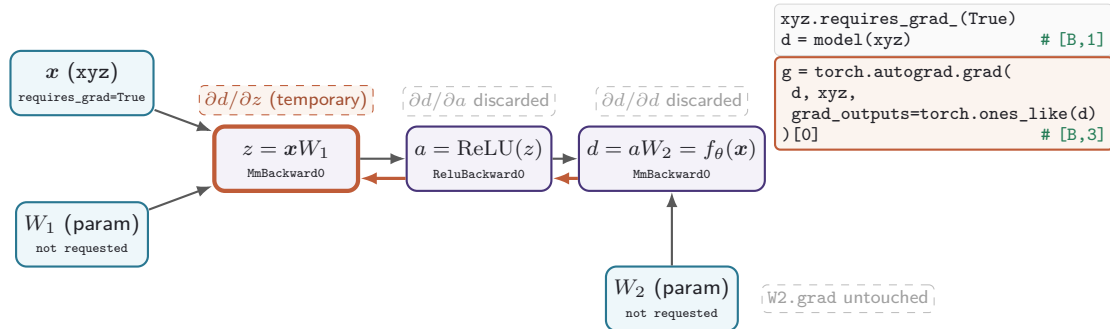
Inference: the gradient of the input leaf xyz



3. Only the path to xyz is followed

`autograd.grad` computes only what is needed to reach the requested input: $\partial d / \partial a = W_2^\top$ is passed on, $\partial d / \partial W_2$ is **skipped**, and `W2.grad` is left untouched.

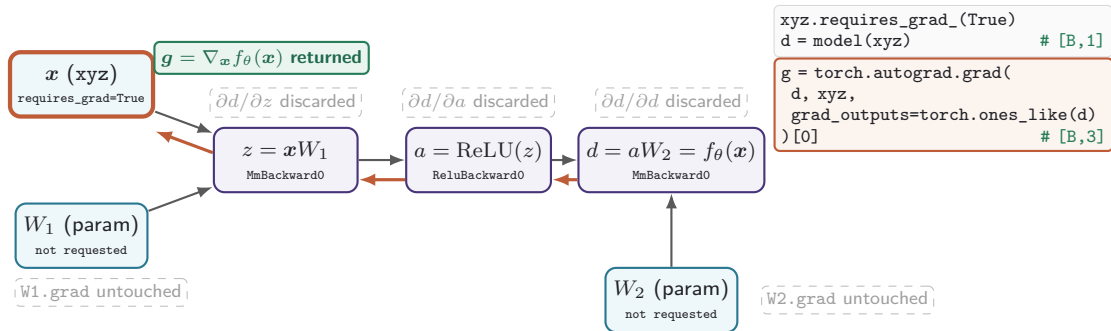
Inference: the gradient of the input leaf xyz



4. Intermediate gradients are still temporary

$\partial d / \partial z = \partial d / \partial a \odot \mathbb{I}[z > 0]$; $\partial d / \partial a$ is discarded, exactly as in training.

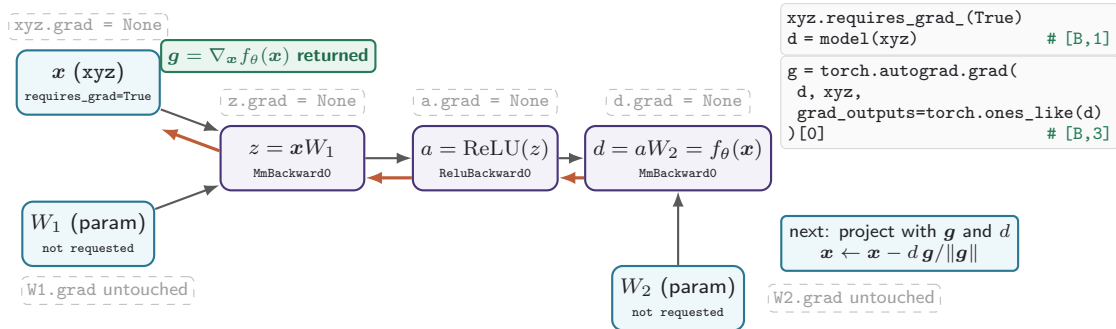
Inference: the gradient of the input leaf xyz



5. Reaching the input leaf

$\partial d / \partial x = \partial d / \partial z W_1^\top = \nabla_x f_\theta(x)$ is **returned** as `g`, a $[B, 3]$ tensor. It is not written to `xyz.grad`, and the parameters are not touched either.

Inference: the gradient of the input leaf xyz



Result: one direction per point, nothing stored

g lives only in our variable: intermediates keep nothing, `xyz.grad` stays `None`, and the graph is freed (`create_graph=False`). d says **how far** and g **which way**: that is all the projection needs.

Using $\nabla_x f_\theta$ to project xyz onto $f_\theta = 0$

```
# one projection step (cf. project_with_field)
```

```
xyz = xyz.detach().requires_grad_(True)
```

```
d = model(xyz) # [B,1]
```

```
g = torch.autograd.grad(  
    d, xyz, grad_outputs=torch.ones_like(d)  
) [0] # [B,3]
```

```
ghat = g / (g.norm(dim=-1, keepdim=True) + eps)
```

```
xyz = (xyz - d * ghat).detach()
```

leaf, requires_grad

x

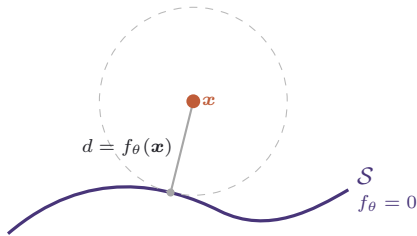


1. Make x a fresh leaf

`detach()` drops the history of the previous step and `requires_grad_(True)` makes `xyz` a **leaf**, so autograd can differentiate with respect to the **input** coordinates.

Using $\nabla_x f_\theta$ to project $\mathbf{xyz}$ onto $f_\theta = 0$

```
# one projection step (cf. project_with_field)
xyz = xyz.detach().requires_grad_(True)
d = model(xyz) # [B,1]
g = torch.autograd.grad(
    d, xyz, grad_outputs=torch.ones_like(d)
)[0] # [B,3]
ghat = g / (g.norm(dim=-1, keepdim=True) + eps)
xyz = (xyz - d * ghat).detach()
```

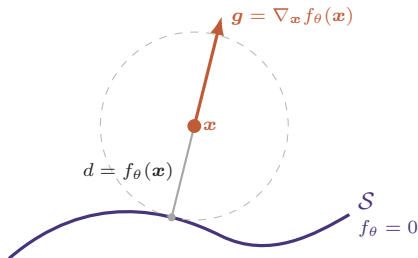


2. Forward pass: how far?

$d = f_\theta(x)$ is the radius of the largest ball around x that does not touch S . It says **how far** to move, but not in which direction.

Using $\nabla_x f_\theta$ to project xyz onto $f_\theta = 0$

```
# one projection step (cf. project_with_field)
xyz = xyz.detach().requires_grad_(True)
d = model(xyz) # [B,1]
g = torch.autograd.grad(
    d, xyz, grad_outputs=torch.ones_like(d)
)[0] # [B,3]
ghat = g / (g.norm(dim=-1, keepdim=True) + eps)
xyz = (xyz - d * ghat).detach()
```

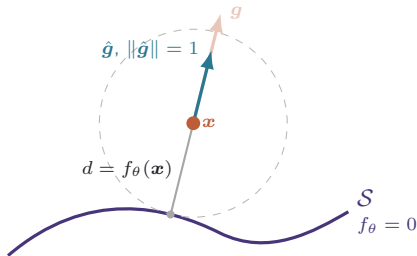


3. Autograd: which way?

autograd.grad walks the graph back from d to x . `grad_outputs=ones_like(d)` is needed because d has B values; since each d_i depends only on x_i , row i of g is $\nabla f_\theta(x_i)$. It points **away** from S .

Using $\nabla_x f_\theta$ to project xyz onto $f_\theta = 0$

```
# one projection step (cf. project_with_field)
xyz = xyz.detach().requires_grad_(True)
d = model(xyz) # [B,1]
g = torch.autograd.grad(
    d, xyz, grad_outputs=torch.ones_like(d)
)[0] # [B,3]
ghat = g / (g.norm(dim=-1, keepdim=True) + eps)
xyz = (xyz - d * ghat).detach()
```

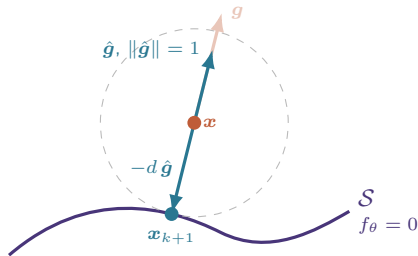


4. Normalize: keep only the direction

A learned field rarely has $\|g\| = 1$. $\hat{g} = g/(\|g\| + \varepsilon)$ keeps the direction, so the step length is set by d alone; ε avoids division by zero where $g \approx 0$.

Using $\nabla_x f_\theta$ to project $\mathbf{x}$ onto $f_\theta = 0$

```
# one projection step (cf. project_with_field)
xyz = xyz.detach().requires_grad_(True)
d = model(xyz) # [B,1]
g = torch.autograd.grad(
    d, xyz, grad_outputs=torch.ones_like(d)
)[0] # [B,3]
ghat = g / (g.norm(dim=-1, keepdim=True) + eps)
xyz = (xyz - d * ghat).detach()
```



5. Step against the gradient

$\mathbf{x}_{k+1} = \mathbf{x} - d\hat{g}$ lands on the **zero level set**: for an exact UDF, the closest point on S . For a learned field we **repeat** (`detach()`, back to step 1) until $f_\theta(\mathbf{x}_k) < \tau$.

Inspecting the gradient field

After computing

$$\mathbf{G} = \nabla_{\mathbf{x}} f_{\theta}(\mathbf{X}) \in \mathbb{R}^{B \times 3},$$

we can inspect magnitude and direction:

```
norm = grad.norm(dim=-1)      # [B]

print("shape:", grad.shape)
print("min norm:", norm.min().item())
print("mean norm:", norm.mean().item())
print("max norm:", norm.max().item())

direction = grad / (
    norm[:, None] + 1e-8
)
```

What to look for?

- ▶ NaNs / Infs;
- ▶ norms close to zero;
- ▶ very large norms;
- ▶ inconsistent directions;
- ▶ abrupt changes near $u = 0$.

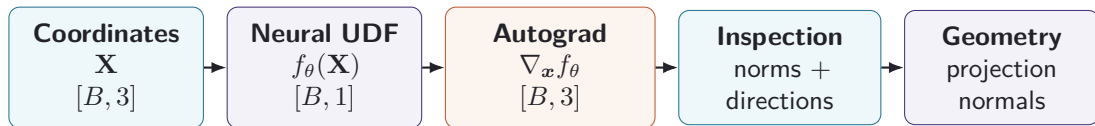
For an ideal distance field

Away from the surface and the **cut locus**, where the distance function is differentiable,

$$\|\nabla_{\mathbf{x}} u(\mathbf{x})\|_2 = 1.$$

In the neural UDF, we check how well this is approximated.

From autograd to geometry



Implementation pattern

`requires_grad` $\rightarrow$ `forward` $\rightarrow$ `autograd.grad` $\rightarrow$ $\nabla_x f_\theta$

Next topic: using distance + gradient to move queries toward the zero level set gradient-based point projection.

The next topic of Block 1 uses the gradients computed here for point projection in the NDF pipeline.

Now to the notebook

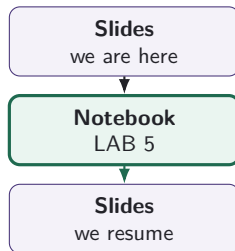
▶ LAB 5 — Spatial gradients of a trained net

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 5 cells 31–36

What we run there

- ▶ `autograd.grad` with respect to the *input*;
- ▶ gradient norms near and far from the surface;
- ▶ the descent field $-\nabla_x f_\theta$ as a quiver plot.



[↩](#) **Back to slide 59**

The intuition: moving a point onto the surface

The intuition: moving a point onto the surface

Suppose a query point

$$\mathbf{x} \in \mathbb{R}^3$$

off the surface, with predicted distance

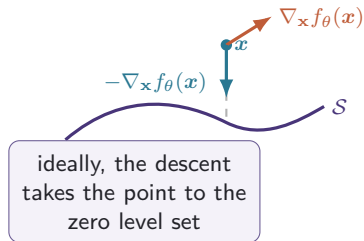
$$f_{\theta}(\mathbf{x}) \approx u(\mathbf{x}).$$

If the field is well learned and we are outside the **cut locus** (unique closest point), then:

- ▶ $f_{\theta}(\mathbf{x})$ tells **how far** we still are;
- ▶ $\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$ tells **the direction** of increase;
- ▶ hence, $-\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$ points along the descent.

The idea of the projection is simple:

walk toward smaller distance until $f_{\theta}(\mathbf{x}) \approx 0$



Why normalize the gradient?

The gradient carries **direction** and **magnitude**.
In the projection, we usually want to separate these two things.

We define the normalized direction:

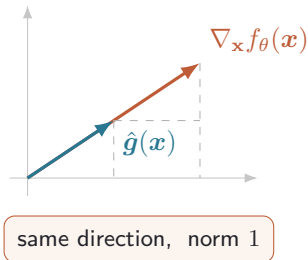
$$\hat{g}(x) = \frac{\nabla_x f_\theta(x)}{\|\nabla_x f_\theta(x)\|_2 + \varepsilon}$$

Then:

- ▶ $\hat{g}(x)$ mainly provides the direction;
- ▶ $f_\theta(x)$ controls the size of the displacement;
- ▶ ε avoids division by zero.

$-\hat{g}(x)$ is the projection direction.

gradient direction + predicted distance $\Rightarrow$ projection step



The projection update in NDF

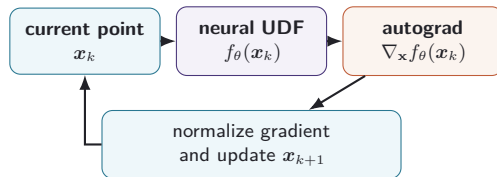
The conceptual form of the update is

$$\mathbf{x}_{k+1} = \mathbf{x}_k - f_{\theta}(\mathbf{x}_k) \frac{\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}_k)}{\|\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}_k)\|_2 + \varepsilon}$$

Interpretation:

- ▶ $f_{\theta}(\mathbf{x}_k)$ estimates the distance to the surface;
- ▶ the normalized gradient chooses the direction;
- ▶ the “−” sign moves the point toward smaller field values.

For an exact UDF away from the surface and from the cut locus, $\|\nabla u\| = 1$ and the ideal step projects onto the closest point. For an approximate neural UDF, iterating and normalizing make the procedure more robust.



Iterative projection

In practice, we project iteratively:

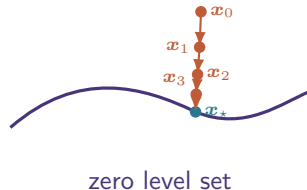
$$\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \mathbf{x}_2 \rightarrow \dots$$

Common stopping criteria:

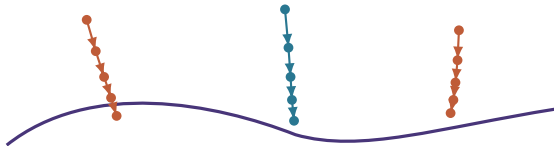
- ▶ $f_{\theta}(\mathbf{x}_k) < \tau$;
- ▶ maximum number of iterations;
- ▶ very small displacement between steps.

A minimal PyTorch version:

```
for k in range(K):  
    x = x.requires_grad_(True)  
    d = model(x)  
    g = torch.autograd.grad(  
        d, x, grad_outputs=torch.ones_like(d)  
    )[0]  
    step = d * g / (g.norm(dim=-1, keepdim=True) + 1e-8)  
    x = (x - step).detach()
```



Visualizing points moving toward the zero level set



$$f_\theta(x) = 0$$

many queries are updated in parallel until they accumulate
on the reconstructed implicit surface

Operational summary

Conceptual pipeline:

$$x_k \xrightarrow{f_\theta} d_k \xrightarrow{\nabla_x f_\theta} \hat{\mathbf{g}}_k \xrightarrow{\text{update}} x_{k+1}$$

Core elements:

- ▶ **closest-surface intuition**: move toward smaller distance;
- ▶ **normalized gradient**: use a stable direction;
- ▶ **iterative projection**: repeat until $f_\theta(x) \approx 0$.

Critical dependency

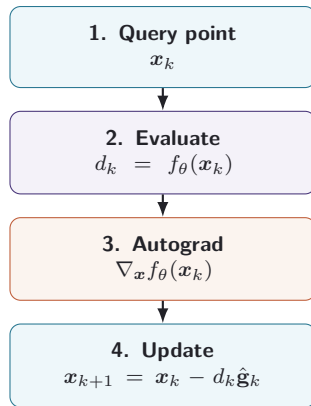
The projection only works well if

$$f_\theta(x) \quad \text{and} \quad \nabla_x f_\theta(x)$$

are geometrically reliable near the surface.

Next topic: sinusoidal activations and SIREN – why the architectural bias also affects the field and its gradients.

Projection combines predicted distance and spatial gradient; NDF applies this idea to its observation-conditioned field $f_X(p)$.



Now to the notebook

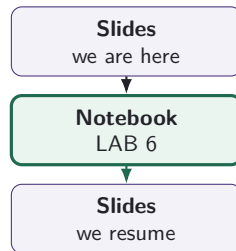
▶ LAB 6 — Gradient-based projection

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 6 cells 37–42

What we run there

- ▶ projection driven by the analytic UDF first;
- ▶ then the same loop driven by the network;
- ▶ step size, iterations and failure modes.



↩ Back to slide 66

Motivation: implicit fields also contain high frequencies

Motivation: implicit fields also contain high frequencies

Coordinate MLPs must represent:

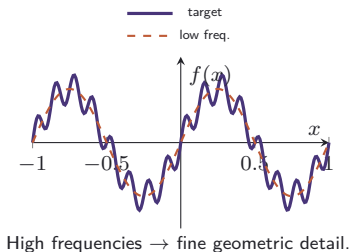
- ▶ fine geometric detail;
- ▶ rapid changes of the field;
- ▶ useful spatial derivatives.

Conventional MLPs often exhibit **spectral bias**:

low-frequency components
are learned first

This can produce:

- ▶ overly smooth fields;
- ▶ loss of small details;
- ▶ less informative gradients.



Rahaman et al., Spectral Bias, ICML 2019; Tancik et al., Fourier Features, NeurIPS 2020; Sitzmann et al., SIREN, NeurIPS 2020.

The sinusoidal activation

A SIREN layer uses

$$\mathbf{h}_{\ell+1} = \sin(\omega_0 (W_\ell \mathbf{h}_\ell + \mathbf{b}_\ell))$$

The parameter ω_0 controls the frequency scale of the activation.

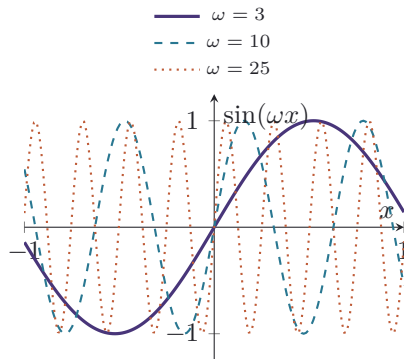
In 1D:

$$\phi(x) = \sin(\omega x).$$

As ω increases:

- ▶ more oscillations fit in the same interval;
- ▶ the network gains more direct access to rapid variations.

ω_0 is not just a numerical detail: it controls the scale of variation available to the layer.



Derivatives: a sine stays a sine/cosine

For a simple activation:

$$\phi(x) = \sin(\omega x),$$

we have

$$\phi'(x) = \omega \cos(\omega x)$$

and

$$\phi''(x) = -\omega^2 \sin(\omega x).$$

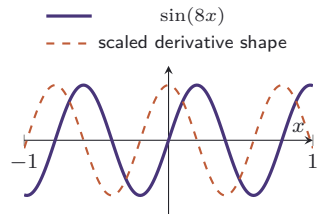
For neural fields, the derivatives:

- ▶ remain smooth;
- ▶ preserve high-frequency structure;
- ▶ are obtained directly through autograd.

For implicit geometry

We want not only $f_\theta(x)$ to be accurate, but also

$$\nabla_x f_\theta(x).$$



Periodic activations provide structured derivatives instead of piecewise-constant slopes.

SIREN requires a specific initialization

Applying $\sin$ in a deep MLP with a generic initialization can produce unstable propagation. SIREN uses an initialization designed to preserve the distribution of the activations across layers.

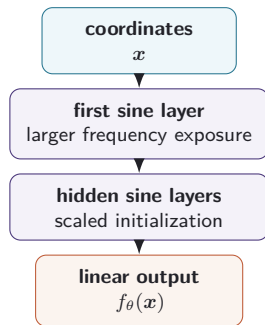
For the first layer:

$$W_0 \sim \mathcal{U} \left(-\frac{1}{n_{\text{in}}}, \frac{1}{n_{\text{in}}} \right)$$

For the hidden layers:

$$W_\ell \sim \mathcal{U} \left(-\frac{\sqrt{6/n_{\text{in}}}}{\omega_0}, \frac{\sqrt{6/n_{\text{in}}}}{\omega_0} \right)$$

SIREN initialization: Sitzmann et al., NeurIPS 2020; official implementation.



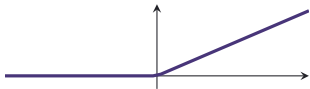
The initialization and ω_0 work together: simply replacing ReLU by $\sin$ is not enough.

ReLU vs. SIREN for implicit fields

ReLU MLP

$$\phi(a) = \max(0, a)$$

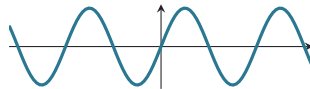
- ▶ piecewise linear function;
- ▶ piecewise constant derivative;
- ▶ simple and robust training;
- ▶ tends to prioritize low-frequency structure;
- ▶ fine details may require more capacity or an encoding.



SIREN

$$\phi(a) = \sin(\omega_0 a)$$

- ▶ smooth and periodic activation;
- ▶ smooth derivatives as well;
- ▶ direct access to high-frequency components;
- ▶ well suited to continuous signals and their derivatives;
- ▶ requires careful initialization and frequency.



For UDF/SDF, the relevant comparison includes the field *and* its derivatives.

Minimal PyTorch implementation

```
class SineLayer(nn.Module):
    def __init__(self, in_f, out_f, w0=30.0):
        super().__init__()
        self.w0 = w0
        self.linear = nn.Linear(in_f, out_f)

    def forward(self, x):
        return torch.sin(
            self.w0 * self.linear(x)
        )
```

```
model = nn.Sequential(
    SineLayer(3, 128, w0=30.0),
    SineLayer(128, 128, w0=30.0),
    SineLayer(128, 128, w0=30.0),
    nn.Linear(128, 1)
)
```

Do not forget

The SIREN architecture includes:

- ▶ a specific initialization;
- ▶ the choice of ω_0 ;
- ▶ a typically linear last layer.

A linear output does not enforce $f_\theta \geq 0$.

Quick experiment

Train ReLU and SIREN on the same queries and compare:

$$|f_\theta - u|, \quad \|\nabla_{\mathbf{x}} f_\theta\|,$$

and the zero level set.

Main message

SIREN changes the bias of the coordinate network in three places:

periodic activation + frequency scale + special initialization

For implicit fields this matters especially because we must simultaneously approximate:

$$f_{\theta}(x) \quad \text{and} \quad \nabla_x f_{\theta}(x).$$

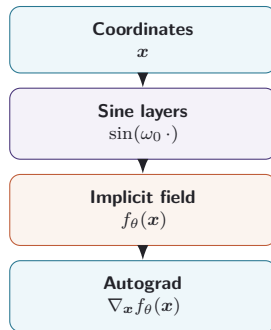
Intuition

ReLU is a general and robust choice.

SIREN was explicitly designed for **continuous signals with high-frequency structure whose derivatives matter**.

Next topic: sampling the field on a grid, applying Marching Cubes and understanding why UDFs do not provide the standard sign change.

Sitzmann et al., Implicit Neural Representations with Periodic Activation Functions, NeurIPS 2020.



Now to the notebook

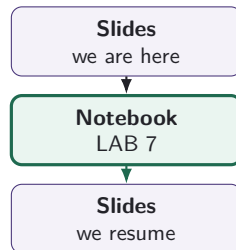
▶ LAB 7 — ReLU vs. SIREN

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 7 cells 43–47

What we run there

- ▶ a minimal `SineLayer` with the SIREN initialization;
- ▶ parameter counts of the two models;
- ▶ a quick head-to-head fit on the same UDF target.



[↩ Back to slide 74](#)

From the continuous field to a grid

From the continuous field to a grid

The network represents a continuous field:

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}.$$

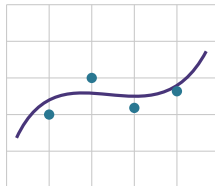
To extract a mesh, we first sample a regular grid:

$$\mathcal{G} = \{\mathbf{x}_{ijk}\}_{i,j,k=1}^R.$$

Then we evaluate

$$V[i, j, k] = f_{\theta}(\mathbf{x}_{ijk}).$$

implicit field



regular samples

Cost

A dense grid has

$$R^3$$

queries: doubling the resolution multiplies the number of evaluations by 8.

continuous field $\rightarrow$ sampled volume $\rightarrow$ mesh

Marching Cubes on a signed field

Marching Cubes extracts an isosurface

$$f(\mathbf{x}) = c$$

by classifying the 8 vertices of each cell as **below** or **above** the isovalue c .

For an SDF and $c = 0$:

$$s_i < 0 \quad \text{or} \quad s_i > 0,$$

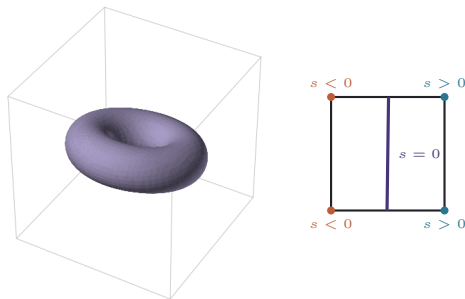
so the classification coincides with the **sign**.

If an edge connects values on opposite sides of the isovalue,

$$(s_i - c)(s_j - c) < 0,$$

there is an interpolated crossing between them. The binary configuration of the corners indexes the case table.

Lorensen & Cline, Marching Cubes, SIGGRAPH 1987.



Left: a plane sweeps the SDF grid of a torus; Marching Cubes builds the mesh behind it, where the slice crosses zero (black curve). **Right:** the 2D analogy of one cell; in 3D the logic is applied to the edges of each cube.

Why is plain Marching Cubes ambiguous for UDFs?

For a UDF:

$$u(\mathbf{x}) \geq 0 \quad \forall \mathbf{x}.$$

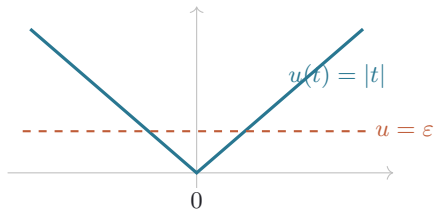
Even when the surface crosses a cell, the values on both sides remain positive:

$$+ \mid u = 0 \mid +.$$

For the isovalue $c = 0$, all UDF samples fall on the same side of the classification ($u_i \geq 0$). In general, the grid does not even sample the zero level set exactly.

Consequence

The standard algorithm does not receive from the UDF the binary opposite-side information it needs to reconstruct the surface directly at $c = 0$.



locally, away from boundaries and from the cut locus,
 $u = \varepsilon > 0$ produces two offset sheets

a UDF provides no inside/outside

Minimal demo with `skimage.measure.marching_cubes`

```
import numpy as np
from skimage import measure

# sdf sampled on a grid whose world-space box is [-1, 1]^3
sdf = field.reshape(R, R, R).cpu().numpy()

bbox_min = np.array([-1.0, -1.0, -1.0])
h = 2.0 / (R - 1)
verts, faces, normals, values = measure.marching_cubes(
    sdf,
    level=0.0,
    spacing=(h, h, h),
)
# spacing rescales indices; it does not add the box origin
verts = verts + bbox_min
```

In `scikit-image`, `marching_cubes` returns vertices, faces, normals and values; the default implementation uses the Lewiner method. Also check the axis convention used to build the grid.

For an SDF

If $\min(s) < 0 < \max(s)$, the isovalue 0 separates corners below/above and defines a natural isosurface.

For a UDF

Since $u \geq 0$, level 0 does not provide the two-sided classification. If the grid never reaches exactly zero, there is no crossing at $c = 0$.

MeshUDF: recovering pseudo-signs from the gradients

MeshUDF keeps the grid, but also evaluates

$$u_i = f_\theta(\mathbf{x}_i), \quad \mathbf{g}_i = \nabla_{\mathbf{x}} f_\theta(\mathbf{x}_i).$$

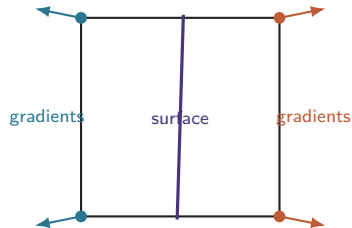
Inside a cell, one corner is used as an anchor and a pseudo signed distance can be built as

$$s_i = \text{sgn}(\mathbf{g}_1^\top \mathbf{g}_i) u_i.$$

Gradients with opposite orientations suggest that the corners are on different sides of the zero level set.

The pseudo-signs then feed the Marching Cubes cases. This is the **local intuition**; the full implementation also propagates orientations across cells (exploration/BFS) and uses voting to reduce inconsistencies caused by noisy gradients.

Guillard, Stella & Fua, MeshUDF, ECCV 2022.



Message

UDF meshing requires recovering information that the unsigned representation removed.

Now to the notebook

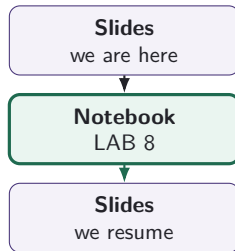
▶ LAB 8 — Marching Cubes: SDF vs. UDF

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 8 cells 48–53

What we run there

- ▶ Marching Cubes at `level=0` on a sphere SDF;
- ▶ the same call on the UDF, and why it fails;
- ▶ a positive level set as an *offset*, not the surface.



↩ Back to slide 80

Where do time and memory disappear?

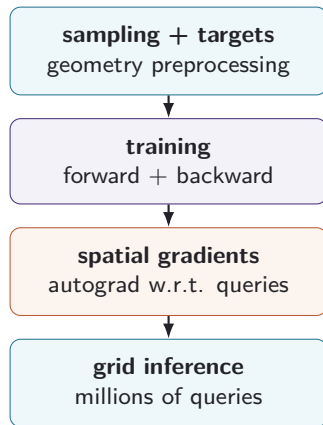
Where do time and memory disappear?

In neural UDFs, the main costs usually show up in:

- ▶ nearest-neighbor distance targets;
- ▶ very large batches of coordinates;
- ▶ the backward pass of the MLP;
- ▶ spatial gradients through autograd;
- ▶ evaluation of R^3 grids;
- ▶ CPU $\leftrightarrow$ GPU transfers.

Rule of thumb

Optimize first what dominates the actual profile, not what *looks* expensive.



1. Normalize coordinates and vectorize the geometry

Normalize the scene to a predictable scale:

$$\mathbf{x}' = \frac{\mathbf{x} - \mathbf{c}}{s},$$

for example, to a bounding box close to

$$[-1, 1]^3.$$

Benefits:

- ▶ a consistent scale for σ , the learning rate and frequencies;
- ▶ better numerical conditioning;
- ▶ hyperparameters reusable across shapes.

Avoid Python loops for geometric operations:

queries $[B, 3]$ vs. points $[M, 3]$.

Use vectorized operations and, when needed, chunks:

$$[B, 3] \rightarrow [B_c, 3] \rightarrow \text{distance kernel}.$$

Careful

A full distance matrix has $B \times M$ elements.

Vectorization without memory control can also be a bottleneck.

2. Mixed precision in training

```
scaler = torch.amp.GradScaler("cuda")

for xyz, target in loader:
    xyz = xyz.to("cuda", non_blocking=True)
    target = target.to("cuda", non_blocking=True)
    opt.zero_grad(set_to_none=True)

    with torch.autocast(
        device_type="cuda",
        dtype=torch.float16
    ):
        pred = model(xyz)
        loss = F.l1_loss(pred, target)

    scaler.scale(loss).backward()
    scaler.step(opt)
    scaler.update()
```

Current PyTorch AMP uses `torch.autocast` and `torch.amp.GradScaler`. For numerically sensitive spatial derivatives, validate the precision and, if needed, compute that step in FP32.

Goal

AMP combines lower-precision operations with operations that stay in higher precision when needed.

Possible benefits

- ▶ less memory;
- ▶ higher throughput on compatible hardware.

Do not assume a speedup

Measure. Small workloads, or workloads dominated by other operations, may not get faster.

3. Do not keep graphs you will not use

For spatial gradients:

$$\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}),$$

use

```
create_graph=True
```

only if a loss needs to differentiate that gradient again.

Otherwise:

```
create_graph=False.
```

After a projection update:

$$\mathbf{x}_{k+1} = \text{detach}(\mathbf{x}_{k+1})$$

before starting the next independent iteration.

Evaluation without gradients

For field/grid inference:

```
torch.inference_mode()
```

avoids building the autograd graph.

Optimizer

```
zero_grad(set_to_none=True)
```

can reduce work and memory compared to explicitly zeroing the buffers in some cases.

Classic symptom

If `memory_allocated()` grows at every iteration, there may be a reference to a previous graph. Growth only in *reserved* memory may come from the caching allocator.

4. Chunked inference and profiling

```
preds = []

with torch.inference_mode():
    for q in grid_xyz.split(131072):
        preds.append(model(q).cpu())

field = torch.cat(preds, dim=0)
```

```
with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
    ],
    record_shapes=True
) as prof:
    train_step(...)
```

Chunking

Choose B_c so as to use the GPU efficiently without running out of memory.

Profiler

Look for:

- ▶ dominant kernels;
- ▶ host/device copies;
- ▶ unexpected shapes;
- ▶ synchronizations;
- ▶ repeated operators.

The PyTorch Profiler collects training/inference metrics and kernel/device activity.

Performance checklist for the notebook

Before optimizing

- ▶ normalize coordinates;
- ▶ make sure batches are large enough;
- ▶ remove unnecessary Python loops;
- ▶ measure memory and time;
- ▶ validate every optimization numerically.

During training

- ▶ AMP when appropriate;
- ▶ `set_to_none=True`;
- ▶ do not retain graphs needlessly;
- ▶ avoid recomputing static targets.

During reconstruction

- ▶ `inference_mode()`;
- ▶ chunked grid;
- ▶ compute gradients only where they are needed;
- ▶ move results to the CPU only when it makes sense.

Common bottleneck in UDFs

It is not always the MLP: nearest-neighbor search, R^3 queries and spatial autograd may dominate the cost.

Now to the notebook

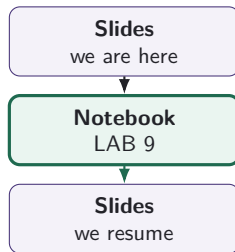
▶ LAB 9 — Where the CPU time goes

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 9 cells 54–59

What we run there

- ▶ `torch.cdist` vs. a KD-tree for distance targets;
- ▶ plain forward vs. `inference_mode`;
- ▶ chunked grid inference and numerical agreement.



[↩](#) **Back to slide 87**

Neural UDF baseline from scratch: the full pipeline

Neural UDF baseline from scratch: the full pipeline



What we are connecting

sampling $\rightarrow$ regression $\rightarrow$ spatial derivatives $\rightarrow$ projection $\rightarrow$ surface reconstruction

Important: this is not the full architecture of the original NDF

This notebook implements a **single-shape coordinate network**. The original NDF takes a sparse point cloud X , voxelizes/encodes X with a multi-scale 3D CNN and queries local features to form the conditioned field

$$f_X(p) = (\Phi \circ \Psi_X)(p).$$

Chibane, Mir & Pons-Moll, NeurIPS 2020, Sec. 3.2.

Step A generating the geometric supervision of the baseline

Input:

$$\mathcal{P} = \{\mathbf{p}_j\}_{j=1}^M.$$

We build queries of three kinds:

$$\mathcal{X} = \mathcal{X}_{surf} \cup \mathcal{X}_{near} \cup \mathcal{X}_{uniform}.$$

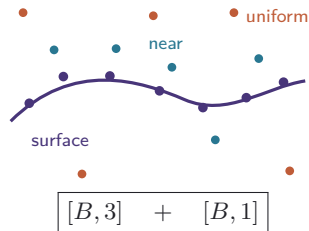
Targets of the point-set baseline:

$$d_i = \min_{\mathbf{p}_j \in \mathcal{P}} \|\mathbf{x}_i - \mathbf{p}_j\|_2.$$

This target is approximate. In the original NDF, $d_i = \text{UDF}(\mathbf{p}_i, S)$ is computed with respect to the ground-truth surface S .

Dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, d_i)\}_{i=1}^N.$$



Step B network, objective and optimizer

Baseline surrogate:

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}, \quad f_{\theta}(\mathbf{x}) \approx u_S(\mathbf{x}).$$

Prediction:

$$\hat{d}_i = f_{\theta}(\mathbf{x}_i).$$

Baseline loss:

$$\mathcal{L}_{L1} = \frac{1}{B} \sum_i |\hat{d}_i - d_i|$$

To get closer to the original NDF, use the truncated version $|\min(\hat{d}_i, \delta) - \min(d_i, \delta)|$ and conditioning on X .

Optimizer:

$$\theta \leftarrow \text{Adam}(\theta, \nabla_{\theta} \mathcal{L}).$$

Minimal architecture

$$3 \rightarrow H \rightarrow H \rightarrow H \rightarrow 1.$$

Activation choice

MLP/ReLU: simple baseline.

SIREN: alternative for representing details and derivatives with periodic activations.

Separate the responsibilities

The loss mainly supervises the distance. Gradient quality is not automatically guaranteed.

Step C training loop + checkpoint

```
model.train()

for step in range(num_steps):
    xyz, udf = sample_batch()

    pred = model(xyz)
    loss = F.l1_loss(pred, udf)  # tutorial baseline

    # Optional: NDF-style truncated target regression
    # delta = truncation_radius  # same units as xyz/udf
    # loss = F.l1_loss(
    #     pred.clamp(max=delta),
    #     udf.clamp(max=delta)
    # )

    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()

    if step % 1000 == 0:
        torch.save({
            "step": step,
            "model": model.state_dict(),
            "optimizer": opt.state_dict(),
        }, "neural_udf_baseline.pt")
```

Save

- ▶ the network parameters;
- ▶ the optimizer state;
- ▶ step/epoch;
- ▶ the relevant configuration.

Monitor

- ▶ train/val L1;
- ▶ field slices;
- ▶ the zero level set;
- ▶ gradients.

Step D evaluating the field and its gradients

For visualization or meshing:

$$\mathbf{X}_G \in \mathbb{R}^{R^3 \times 3}.$$

We evaluate:

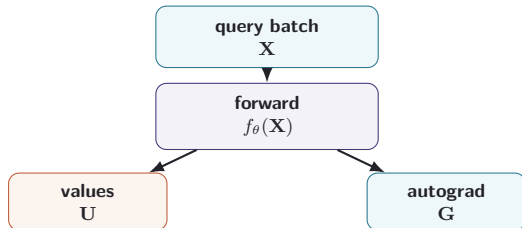
$$\mathbf{U} = f_{\theta}(\mathbf{X}_G).$$

For projection:

$$\mathbf{G} = \nabla_{\mathbf{X}_G} f_{\theta}(\mathbf{X}_G).$$

The two outputs play different roles:

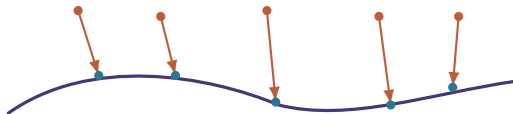
$\mathbf{U} \Rightarrow$ distance/surrogate, $\mathbf{G} \Rightarrow$ local direction.



Step E projecting a dense cloud onto the surface

For each initial point $\mathbf{x}_0$, we iterate. The update is geometrically justified when the field approximates a UDF in a differentiable region, away from the cut locus:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - f_{\theta}(\mathbf{x}_k) \frac{\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}_k)}{\|\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}_k)\|_2 + \varepsilon}.$$



implicit zero level set

Result:

$$\mathcal{Q}_{proj} = \{\mathbf{x}_i^*\},$$

a dense point cloud close to the learned surface.

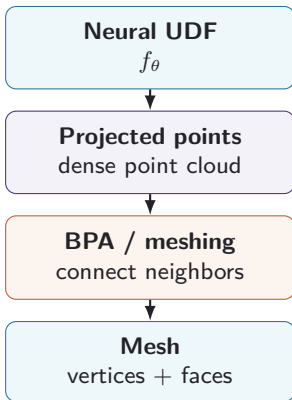
Step F explicit reconstruction

The **original NDF** extracts a dense surface point cloud by projection and can triangulate that cloud. The tutorial baseline reuses the same geometric post-processing idea.

Pipeline:

$$f_{\theta} \rightarrow \mathcal{Q}_{proj} \rightarrow \text{triangulation} \rightarrow \mathcal{M}$$

In the original pipeline, Ball-Pivoting is used to triangulate the surface cloud.



NDF extracts dense surface samples by gradient-based projection; the original reconstruction pipeline triangulates them with Ball-Pivoting.

End-to-end baseline: what should work before moving on

Data

- ▶ normalized point cloud;
- ▶ surface / near / uniform queries;
- ▶ targets consistent with the geometry (surface UDF, when available; nearest-point only as an approximation).

Learning

- ▶ MLP or SIREN;
- ▶ L1 regression; optionally, NDF-style truncation;
- ▶ optimizer;
- ▶ checkpoints;
- ▶ a converging loss.

Geometry

- ▶ coherent field slices;
- ▶ numerically stable $\nabla_x f_\theta$;
- ▶ projection converges;
- ▶ the projected cloud covers the surface;
- ▶ explicit reconstruction preserves detail.

The critical question

If the distance looks good but the reconstruction fails:

inspect the gradients first.

This pipeline is the conceptual baseline for understanding why the UDF literature evolved.

Now to the notebook

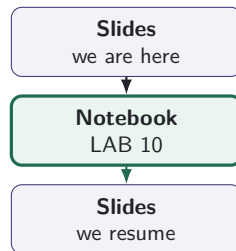
▶ LAB 10 — End-to-end Neural UDF in 3D

File: `sibgrapi_udf_cpu_tutorial.ipynb`

Notebook block: LAB 10 cells 60–76

What we run there

- ▶ Steps A–F of the baseline, in order;
- ▶ open 3D patch, KD-tree targets, training, slices;
- ▶ gradient check, projection and checkpoint.



↩ **Back to slide 96**

Block 1: what NDF demonstrated

Block 1: what NDF demonstrated

The original NDF learns an **observation-conditioned** field:

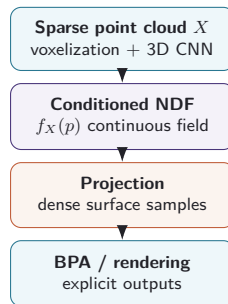
$$f_X(p) = (\Phi \circ \Psi_X)(p) : \mathbb{R}^3 \rightarrow \mathbb{R}_{\geq 0},$$

where X is the input sparse point cloud, Ψ_X queries multi-scale features extracted by a 3D CNN and Φ regresses the unsigned distance.

This enables:

- ▶ reconstruction conditioned on sparse point clouds;
- ▶ open surfaces and non-watertight geometry;
- ▶ continuous queries;
- ▶ spatial gradients through backprop;
- ▶ dense surface sampling by projection.

Chibane, Mir & Pons-Moll, NeurIPS 2020, Secs. 3.2–3.4.



The baseline and NDF leave open problems

Learning / supervision

- ▶ NDF relies on ground-truth UDF for training; the point-set baseline uses approximate targets;
- ▶ sparse/noisy point clouds remain hard;
- ▶ a correct distance does not guarantee a correct gradient.

Reconstruction

- ▶ a UDF provides no sign changes for standard MC;
- ▶ projection may produce fragmentation;
- ▶ downstream triangulation adds another stage.

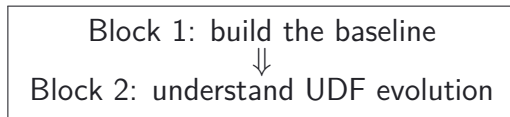
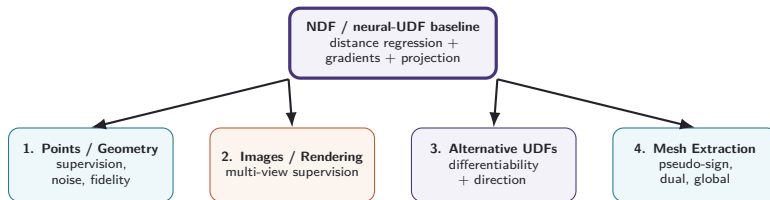
Near-surface behavior

- ▶ $u(t) = |t|$ is not differentiable at 0;
- ▶ the neural surrogate may round off or distort the cusp;
- ▶ projection depends on the learned gradient.

Observations beyond points

- ▶ how do we learn from images?
- ▶ how do we integrate rendering?
- ▶ how do we exploit modern geometry priors?

The limitations of NDF become the agenda of Block 2



Taxonomy of the companion survey of this tutorial (to appear): Points/Geometry, Images/Rendering, Alternative Representations, and Mesh Extraction/Meshing.

Coordinate networks and spectral bias

- G. Cybenko, *Approximation by Superpositions of a Sigmoidal Function*, MCS, 1989.
- K. Hornik, M. Stinchcombe, H. White, *Multilayer Feedforward Networks Are Universal Approximators*, Neural Networks, 1989.
- N. Rahaman et al., *On the Spectral Bias of Neural Networks*, ICML, 2019.
- M. Tancik et al., *Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains*, NeurIPS, 2020.
- V. Sitzmann et al., *Implicit Neural Representations with Periodic Activation Functions (SIREN)*, NeurIPS, 2020.

Implicit representations and neural fields

- L. Mescheder et al., *Occupancy Networks: Learning 3D Reconstruction in Function Space*, CVPR, 2019.
- J. J. Park et al., *DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation*, CVPR, 2019.
- B. Mildenhall et al., *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*, ECCV, 2020.

Unsigned distance fields

- J. Chibane, A. Mir, G. Pons-Moll, *Neural Unsigned Distance Fields for Implicit Function Learning (NDF)*, NeurIPS, 2020.
- J. Zhou et al., *Learning a More Continuous Zero Level Set in Unsigned Distance Fields through Level Set Projection*, ICCV, 2023.
- M. Fainstein, V. Siless, E. Iarussi, *DUDF: Differentiable Unsigned Distance Fields with Hyperbolic Scaling*, CVPR, 2024.

Surface extraction and meshing

- W. E. Lorensen, H. E. Cline, *Marching Cubes: A High Resolution 3D Surface Construction Algorithm*, SIGGRAPH, 1987.
- F. Bernardini et al., *The Ball-Pivoting Algorithm for Surface Reconstruction*, TVCG, 1999.
- T. Lewiner et al., *Efficient Implementation of Marching Cubes' Cases with Topological Guarantees*, Journal of Graphics Tools, 2003.
- B. Guillard, F. Stella, P. Fua, *MeshUDF: Fast and Differentiable Meshing of Unsigned Distance Field Networks*, ECCV, 2022.

■ Companion survey of this tutorial

M. L. de L. Bessa, *Beyond Watertight Geometry: Neural Unsigned Distance Fields*, SIBGRAPI 2026 — to appear.